126198

Full Name: Virag Koradiya

Roll Number: 202412040

IT628: Systems Programming, Winter 2024-25
First In-Sem Exam (1 hour)

Feb 10, 2025

Instructions:

- Make sure your exam is not missing any sheets, then write your name and roll number on the top of this page.
- Clearly write your answer in the space indicated. None of the questions need long answers.
- For rough work, do not use any additional sheets. Rough work will not be graded.
- The exam has a maximum score of 16 points. It is CLOSED BOOK. Notes are NOT allowed.
- Anyone who copies or allows someone to copy will receive F grade.

Problem 1 (/6):	6
Problem 2 (/4):	4
Problem 3 (/2):	2
Problem 4 (/2):	2
Problem 5 (/2):	0
TOTAL (/16):	14

Problem 1. (6, 1.5 each points):

Circle the *single best* answer to each of the following questions. You will get -0.5 points for a wrong answer. If you circle more than one answer, you will lose the mark for the corresponding question.
Assume that all system calls execute without error. Additionally, assume that each call to printf flushes its output just before it returns.

1. When it succeeds, `execve` is called once and returns how many times?

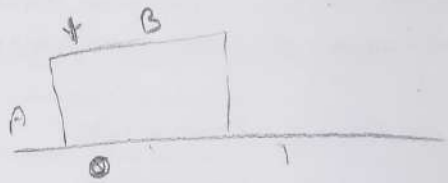
   (a) 0
   (b) 1
   (c) 2
   (d) 3

2. A program blocks `SIGCHLD` and `SIGUSR1`. It is then sent a `SIGCHLD`, a `SIGUSR1`, and another `SIGCHLD`, in that order. What signals does the program receive after it unblocks both of those signals (you may assume the program does not receive any more signals after)?

   (a) None, since the signals were blocked they are all discarded.
   (b) Just a single `SIGCHLD`, since all subsequent signals are discarded.
   (c) Just a single `SIGCHLD` and a single `SIGUSR1`, since the extra `SIGCHLD` is discarded.
   (d) All 3 signals, since no signals are discarded.

3. Consider the following program.

```
int main()
{
    int status;
    pid_t pid;
    printf("A ");
    pid = fork();
    printf("%d ", !pid);
    if (wait(&status) > 0) {
        if (WIFEXITED(status))
            printf("%d ", WEXITSTATUS(status));
        exit(0);
    }
    printf("B ");
    exit(1);
}
```



Among the following, which one cannot be printed during an execution of the program?

   (a) A 1 B 0 1
   (b) A 1 B 1 0
   (c) A 1 0 B 1
   (d) A 0 1 B 1

202412040

4. A process exists in the zombie (also known as defunct) state because:

   (a) It is running but making no progress.
   (b) The user may need to restart it without reloading the program.
   (c) The parent may need to read its exit status.
   (d) The process may still have children that have not exited.

1.5

Problem 2. (4 points):

When creating programs to automatically test student assignments, we want to detect if a student's program gets into an infinite loop. Complete the program below that takes the name of a second program to run as a command line argument. The program will create a process to execute the second program (which itself takes no arguments). If the second program does not terminate after 10 seconds, the original process will terminate it. The program will print out an error message if it must terminate the second process.

```
int main(int argc, char *argv[]) {
    int pid;

    if ((pid = fork()) == 0) {
        execl(argv[1], argv[1], 0);
    }
    else {
        sleep(10);
        if (waitpid(pid, NULL, WNOHANG) == 0) {        2
            printf("Not Terminated Yet");              1
            kill(Pid, SIGINT);                         1
        }
    }
}
```

2024120040

(2)

Problem 4. (2, 1 each points):

You are asked to show what each of the following two programs prints as output.

- Assume that file `infile.txt` contains the ASCII text characters "15213";
- You may assume that system calls do not fail;
- When a process with no children invokes `waitpid(-1, NULL, 0)`, this call returns immedia

Each of the following questions has a unique answer.

Program A

```
int main()
{
    int fd;
    char c;

    fd = open("infile.txt", O_RDONLY, 0);

    fork();
    waitpid(-1, NULL, 0);

    read(fd, &c, sizeof(c));
    printf("%c", c);

    return 0;
}
```

(1)

OUTPUT: 15 ✓

Program B

```
int main()
{
    int fd;
    char c;

    fork();
    waitpid(-1, NULL, 0);

    fd = open("infile.txt", O_RDONLY, 0);

    read(fd, &c, sizeof(c));
    printf("%c", c);

    return 0;
}
```

(1)

OUTPUT: 11 ✓

(2)

Problem 3. (2, 1 each points):
For problems A-B, indicate how many "hello" output lines the program would print. (For space reasons, we are not checking error return codes, so assume that all functions return normally.) *Caution: Don't overlook the* `printf` *function in* `main`.

Problem A

```
void doit() {
  if (fork() == 0) {
    fork();
    printf("hello\n");
    exit(0);
  }
  return;
}

int main() {
  doit();
  printf("hello\n");
  exit(0);
}
```

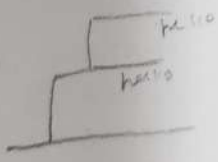
Answer: 3 output lines.

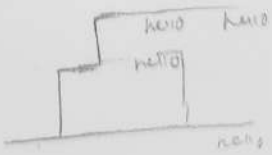
Problem B

```
void doit() {
  if (fork() == 0) {
    fork();
    printf("hello\n");
    return;
  }
  return;
}

int main() {
  doit();
  printf("hello\n");
  exit(0);
}
```

Answer: 5 output lines.





Problem 5. (2 points):

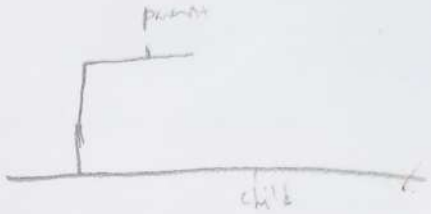
Consider the C program below. For space reasons, we are not checking error return codes. Assume that all functions return normally and that the proper header files have been included.

```
void handler1(int sig) {
    printf("child got SIGUSR1\n");
}

void handler2(int sig) {
    printf("parent got SIGUSR1\n");
}

int main() {
    pid_t pid;

    if ((pid = fork()) == 0) {
        signal(SIGUSR1, handler1);
        kill(getppid(), SIGUSR1);
        sleep(1);
        exit(0);
    }
    else {
        signal(SIGUSR1, handler2);
        kill(pid, SIGUSR1);
        waitpid(pid, NULL, 0);
    }
    return 0;
}
```



The programmer who wrote this program would like it to always print these two lines:

```
parent got SIGUSR1
child got SIGUSR1
```

These lines could appear in reverse order in different executions because the scheduler decides whether the parent or the child runs first following a fork. However, this program has a bug—it does not print the two lines the programmer expects it to print. Identify **and briefly explain** the bug. Do **NOT** try to identify style flaws.

The bug is in kill(). from
It should. The kill in child is should be in th

The bug is SIGUSR1, there should be two different

SIGUSR.